

Problem B: Party Investigation

Time limit: 1 second

The recent election battle in the animal kingdom was heated like never before. In the end the lion won the office of president once again in an incredibly close victory over the giraffe. Now you as reporter want to investigate the aftermath in the population.

For your latest article you are interested how relationships between direct neighbours have changed if one neighbour has voted for the lion and the other for the giraffe. As all animals are interested in politics, you know that every single one voted either one candidate or the other. As research ground you chose the longest street in the animal kingdom. Certainly there must be two citizens who voted for different candidates. And sure enough, the animal in house number 1 voted for the lion and the animal in house number n voted for the giraffe.

Now you know: Somewhere in the street must at least be one edge between lion voters and giraffe voters, but where? You want to publish the article as soon as possible and the street has so many houses that you don't really have time to ask every animal what they voted for.

Interaction

This is an interactive problem. Your submission will be run against an *interactor*. This means that the output of your submission is sent to the interactor and the output of the interactor is sent to the standard input of your submission. This interaction must follow a specific protocol:

First the interactor sends one line containing a single integer n ($2 \leq n \leq 10^9$), the highest house number of the street.

After that your submission must send requests of the following two types:

- One line of the form “? q ”, where q ($1 \leq q \leq n$) is an integer. The interactor will respond “lion”, if the animal in house number q voted for the lion, and “giraffe” otherwise.
- One line of the form “! e ”, where the animal in house number e voted one candidate but the animal in house number $e + 1$ voted the other candidate.

After every request you should *flush* the standard output to ensure that the request is sent to the interactor. For example, you can use `fflush(stdout)` in C++, `System.out.flush()` in Java, `sys.stdout.flush()` in Python, and `hFlush stdout` in Haskell.

Your submission may send as many requests of the first type as you want, as long it does not exceed the time limit.

Your submission must send exactly one request of the second type. After sending this request, it must exit with exit code 0 as usual.

Interaction is adaptive; the interactor may decide the votes of the street's residents on the fly based on the queries that you send, but will always do so in a way that is consistent with previous queries and responses.

Your submission will be accepted if it follows the protocol above and it reports any correct edge between lion and giraffe voters. If it sends any invalid request or reports an incorrect edge, it will be judged as “Wrong Answer”.

Note that sending a query for house number 1 always returns “lion” and a query for house number n always returns “giraffe”.

A testing tool is provided to help you develop your solution.

Read

Sample Interaction 1

Write

5

? 4

giraffe

? 3

lion

! 3

Read

Sample Interaction 2

Write

11

? 4

giraffe

? 5

giraffe

? 3

giraffe

? 1

lion

? 11

giraffe

? 6

lion

! 5